Icindekiler

1 Projenin Ozeti **3**

2 Projenin Amaci ve Kapsami **3**

3 Genel Uygulama Akisi **4**

4 Proje Dizin Yapisi **5**

5 Projeyi Nasil Ilerlettik **6**

6 Firebase Entegrasyonu ve Veri Katmani **6**

7 Kullanici Hesabi ve Oturum Yonetimi **7**

8 Yonetici (Admin) Paneli ve Siparis Yonetimi **8**

9 Kayitlarin Saklanmasi ve Verinin Korunmasi **8**

10 Kullanici Deneyimi Tarafi **9**

11 Arayuz Tarafi **9**

12 Kod Bolumleri **9**

12.1 Dosya: pubspec.yaml 10

12.2 Dosya: lib/firebase_options.dart 10

12.3 Dosya: lib/app/app_config.dart 10

12.4 Dosya: lib/main.dart 11

12.5 Dosya: lib/screens/auth_page.dart 12

12.6 Dosya: lib/data/books_repository.dart 16

12.7 Dosya: lib/data/user_repository.dart 17

12.8 Dosya: lib/data/admin_repository.dart 19

12.9 Dosya: lib/app/admin_app.dart 21

12.10 Dosya: lib/screens/admin_order_detail_page.dart 29

12.11 Dosya: lib/screens/admin_add_book_page.dart 35

12.12 Dosya: firestore.rules 40

12.13 Dosya: lib/app/book_store_app.dart 40
12.14 Dosya: lib/screens/cart_tab.dart 46
12.15 Dosya: lib/models/order_record.dart 46

13 Sonuc ve Genel Degerlendirme **47**

14 Ekran Goruntuleri **48**

1. Projenin Ozeti

Bu rapor, Mobil Programlama dersi kapsaminda gelistirdigimiz `kitap_kiralama` projesini anlatmak icin hazirlanmistir. Proje adinda kiralama ifadesi bulunsa da mevcut uygulama akisi kitaplari listeleyen, detayini gosteren, sepete ekleyen ve siparis gecmisini takip ettiren bir mobil arayuz uzerine kurulmustur.

Bu projede amacimiz kitap odakli bir mobil uygulama deneyimini final projesi seviyesinde duzenli, temiz ve sunulabilir bir yapiyla ortaya koymakti. Uygulama kullaniciya once kitap katalogunu sunuyor, ardindan kitap detayina inmesine izin veriyor, daha sonra sepet uzerinden siparis verme akisini tamamlatiyor. Verilen siparisler de ayri bir ekranda tutuluyor ve detayli olarak gorulebiliyor.

Projenin son surumunde uygulama yalnizca bellekte calisan bir prototip olmaktan cikarildi ve Firebase tabanli bir veri katmaniyla desteklendi. Boylece kitap katalogu Cloud Firestore uzerinden cekilmeye, kullanici girisi Firebase Authentication uzerinden yapilmaya, sepet ve siparis kayitlari da her kullaniciya ozel olarak Firestore icinde tutulmaya baslandi. Buna ek olarak yonetici paneli eklendi; admin hesabi tum siparisleri gorup kargo durumunu degistirebiliyor ve yeni kitap ekleyebiliyor.

2. Projenin Amaci ve Kapsami

Bu projenin temel amaci sadece bir liste ekrani gostermek degildi. Biz burada daha butunlu bir deneyim kurmak istedik:

1. Kullanici uygulamayi acinca once bir oturum acma ekrani ile karsilassin.
2. Hesabi yoksa hemen ayni ekrandan kayit olabilsin.
3. Giris yaptiktan sonra kitaplari iki sutunlu bir duzende gorebilsin.
4. Arama ve filtreleme ile liste icinde rahat gezebilsin.
5. Bir kitabin detayina girip icerigi, konusu ve temel bilgilerini okuyabilsin.
6. Begendigi kitabi sepete ekleyebilsin ve sepet veritabaninda saklansin.
7. Siparis verince bu kayit Firestore tarafinda kalici olarak tutulsun.
8. Gecmis siparislerden birine tiklayinca o siparisin detaylarini gorebilsin.
9. Admin hesabi ile giris yapildiginda musteri uygulamasi yerine yonetici paneli acilsin.
10. Yonetici tum siparisleri tek ekranda gorup kargo durumunu degistirebilsin.
11. Yonetici yeni kitap ekleyebilsin ve magazayi onizleme modunda gezebilsin.

Bu kapsam icinde proje, ders duzeyinde yeterli olacak kadar islevsel; ama bilincli olarak fazla karmasik olmayan bir noktada tutuldu. Yine de rapor teslim asamasinda kod yapisi daha okunur ve savunulabilir olsun diye dosya yapisi profesyonel sekilde duzenlendi ve gercek bir veri katmani eklendi.

3. Genel Uygulama Akisi

Uygulamanin genel akisi su sekilde ilerliyor:

1. Uygulama acildiginda Firebase baslatiliyor ve oturum durumu kontrol ediliyor.
2. Oturum yoksa giris / kayit ekrani aciliyor.
3. Kullanici e-posta ve sifre ile giris yapiyor ya da yeni hesap olusturuyor.
4. Giris yapan e-posta `admin@gmail.com` ise yonetici paneli aciliyor.
5. Normal kullanici girisinde ana ekranda kitap kartlari Firestore'dan cekilip listeleniyor.
6. Ust taraftaki arama alani ile kitap, yazar veya kategori bazli arama yapabiliyor.
7. Filtre butonu ile kategori secimi ve siralama tercihi aciliyor.
8. Kartlardan birine tiklaninca detay sayfasi aciliyor.
9. Detay sayfasinda kitap ozetine, neden okunmasi gerektigine ve kitap bilgilerine ulasilabiliyor.
10. `Sepete Ekle` butonu ile kitap kullanicinin Firestore sepetine kaydediliyor.
11. `Sepet` sekmesinde secilen kitaplar listeleniyor ve toplam tutar gosteriliyor.
12. `Siparis Ver` sonrasi siparis kaydi `users/{uid}/orders` koleksiyonuna `Onay Bekliyor` durumuyla yaziliyor ve sepet temizleniyor.
13. Gecmis siparislerden birine tiklaninca siparis detay sayfasi aciliyor.
14. Admin panelinde tum kullanicilarin siparisleri listeleniyor, siparis detayi acilip durum combobox ile guncelleniyor.
15. Admin panelindeki `Kitap Ekle` sekmesinden `books` koleksiyonuna yeni kitap yazilabiliyor.
16. Sag ust kosedeki cikis butonu ile oturum kapatilabiliyor.

Bu akis teknik olarak buyuk bir e-ticaret sistemi olmasa da final projesi anlatimi icin yeterli bir butunluk sagliyor.

4. Proje Dizin Yapisi

Bu asamada proje yapisi daha mantikli klasorlere ayrildi ve Firebase ile ilgili dosyalar eklendi:

```
kitap_kiralama/
|-- assets/
|   `-- covers/
|-- lib/
|   |-- app/
|   |   |-- admin_app.dart
|   |   |-- app_config.dart
|   |   `-- book_store_app.dart
|   |-- data/
|   |   |-- admin_repository.dart
|   |   |-- demo_books.dart
|   |   |-- books_repository.dart
|   |   `-- user_repository.dart
|   |-- models/
|   |   |-- book.dart
|   |   `-- order_record.dart
|   |-- screens/
|   |   |-- admin_add_book_page.dart
|   |   |-- admin_order_detail_page.dart
|   |   |-- auth_page.dart
|   |   |-- book_detail_page.dart
|   |   |-- cart_tab.dart
|   |   |-- order_detail_page.dart
|   |   |-- orders_tab.dart
|   |   `-- store_home_tab.dart
|   |-- theme/
|   |   |-- app_palette.dart
|   |   `-- app_theme.dart
|   |-- widgets/
|   |   |-- book_card.dart
|   |   |-- book_cover.dart
|   |   |-- cart_icon_badge.dart
|   |   |-- detail_info_tile.dart
|   |   |-- detail_section_card.dart
|   |   |-- detail_stat_card.dart
|   |   |-- highlight_tag.dart
|   |   |-- order_status_pill.dart
|   |   `-- summary_row.dart
|   |-- firebase_options.dart
|   `-- main.dart
|-- firestore.rules
|-- firebase.json
```

```
|-- .firebaserc
|-- reports/
|   |-- kitap_kiralama_raporu.md
|   |-- kitap_kiralama_raporu.tex
|   |-- kitap_kiralama_raporu.pdf
|   `-- screenshot_01.png ... screenshot_10.png
`-- pubspec.yaml
```

Buradaki en onemli farklar Firebase ayarlari, auth ekrani, veri repository'leri ve admin paneli dosyalarinin eklenmis olmasidir. Ayni zamanda proje koku artik `firestore.rules`, `firebase.json` ve `.firebaserc` ile birlikte bir Firebase projesi olarak da yapilandirilmistir.

5. Projeyi Nasil Ilerlettik

Projeyi gelistirirken asama asama ilerledik. Ilk adimda arayuz kurgusu Figma uzerinde cizildi. Ana ekran, kitap kartlari, detay sayfasi, sepet ve alt menu yapisi once tasarim olarak netlestirildi.

Ikinci adimda bu tasarim Flutter tarafina aktarildi. Yani Figma'da dusundugumuz sayfa duzeni, renk dili ve kart yapisi Flutter widgetlari ile gercek uygulama ekranlarina donusturuldu.

Ucuncu adimda ise kullanicinin gordugu ekranlara islev kazandirildi. Bu kisimda butonlarin calismasi, menu gecisleri, kitap detayina gitme, sepete ekleme, siparis olusturma ve siparis kayitlarini listeleme gibi arka plan mantigi eklendi. Bu noktada veriler henuz yalnizca bellekte tutuluyordu.

Dorduncu adimda projeyi tek dosyali yapidan cikartip `app`, `screens`, `models`, `data`, `theme` ve `widgets` klasorlerine ayirdik. Boylece refactor sonrasi kod tabani daha okunabilir hale geldi.

Besinci adimda Firebase entegrasyonu yapildi. `kitap-kiralama-app` adinda bir Firebase projesi olusturuldu, FlutterFire CLI ile yapilandirma dosyalari uretildi, Firestore veritabani acildi, kitap verileri Firestore'a tasindi, Authentication acilarak e-posta sifre giris destegi eklendi ve sepet ile siparisler kullanici bazli olarak Firestore icinde saklanmaya baslandi.

Son adimda ise yonetici paneli ve siparis durum yonetimi eklendi. `admin@gmail.com` hesabi normal magaza yerine admin paneline yonlendirildi; admin tum siparisleri tek listede gormeye, siparis durumunu degistirmeye, yeni kitap eklemeye ve magazayi onizleme modunda gezmeye basladi.

6. Firebase Entegrasyonu ve Veri Katmani

Projenin son asamasinda yalnizca arayuz yeterli olmaz dusuncesiyle bir bulut veri katmani eklenmesine karar verildi. Bu kapsamda asagidaki adimlar yapildi:

1. Firebase CLI uzerinden `kitap-kiralama-app` projesi olusturuldu.
2. Gerekli Firestore ve Authentication API'leri etkinlestirildi.
3. `(default)` Firestore veritabani `europe-west1` bolgesinde olusturuldu.
4. FlutterFire CLI ile `lib/firebase_options.dart` dosyasi olusturularak Android, iOS ve Web icin yapilandirma uretildi.
5. `firebase_core`, `firebase_auth` ve `cloud_firestore` paketleri eklendi.
6. Kitap verileri seed scripti ile Firestore'da `books` koleksiyonuna yazildi.
7. `firestore.rules` dosyasi ile guvenlik kurallari yazilip Firestore'a deploy edildi.
8. Firebase konsolundan e-posta sifre saglayicisi aktiflestirildi ve test hesabi olusturuldu.
9. Admin hesabi icin admin kullanicisi hazirlandi.

Firestore tarafindaki veri modeli su sekildedir:

- `books/{bookId}` koleksiyonu kitap dokumanlarini tutar.
- `users/{uid}/state/cart` dokumani kullanicinin sepetindeki kitaplarin id listesini tutar.
- Her kullanicinin `orders` alt koleksiyonu siparislerini tutar; her sipariste kod, kitap id listesi, toplam fiyat, tarih, durum, kullanici id'si ve e-posta alanlari bulunur.

Guvenlik kurallarinda kitap koleksiyonu giris yapmis kullanicilar tarafindan okunabiliyor, yazma yetkisi ise sadece admin hesabina veriliyor. Kullanici altindaki dokumanlara sahibi olan kullanici erisebiliyor. Ayrica admin, tum kullanicilarin siparislerini okuyabiliyor ve siparis durumunu guncelleyebiliyor.

7. Kullanici Hesabi ve Oturum Yonetimi

Uygulamada oturum yonetimi Firebase Auth akisina bakilarak yapiliyor. `main.dart` icindeki kontrol; oturum yoksa `AuthPage` ekranini, oturum admin hesabina aitse `AdminApp` panelini, diger kullanicilarda ise `BookStoreApp` kabugunu render ediyor.

`AuthPage` ekrani tek bir form uzerinden hem giris hem kayit isini yapacak sekilde tasarlandi:

- Kullanici "Giris Yap" modunda iken `signInWithEmailAndPassword` cagrisi yapiliyor.
- "Kayit Ol" modunda iken `createUserWithEmailAndPassword` cagrisi yapiliyor.
- `FirebaseAuthException` durumlari yakalanip kullanici dostu bir hata mesaji olarak ekranda gosteriliyor.

Test sunumu kolay olsun diye form alanlari acilirken test hesabi bilgileriyle dolu geliyor. Uygulama icindeki cikis butonu Firebase Auth uzerinden oturumu kapatiyor.

8. Yonetici (Admin) Paneli ve Siparis Yonetimi

Yonetici ozellikleri sade tutuldu ve ayri bir API katmani yazilmadi. Admin hesabi olarak `admin@gmail.com` kullanildi. `main.dart` icindeki oturum kontrolu artik uc yonlu calisiyor: oturum yoksa giris ekrani, giris yapan kullanici admin ise admin paneli, normal kullanici ise magaza uygulamasi aciliyor.

`AdminApp` uc sekmeli bir panel olarak tasarlandi:

- `Siparisler` sekmesi tum kullanicilarin siparislerini tek ekranda gosteriyor.
- `Magaza` sekmesi kitap listesini admin icin onizleme modunda actiriyor.
- `Kitap Ekle` sekmesi yeni kitap bilgilerini alip Firestore `books` koleksiyonuna yaziyor.

Siparis durumu akisi onay bekliyor, hazirlaniyor, kargoya verildi ve teslim edildi adimlarindan olusuyor. Yeni verilen siparisler ilk olarak `Onay Bekliyor` durumu ile kaydediliyor. Admin siparis detayina girdiginde kargo durumunu combobox ile degistirebiliyor. Bu degisiklik dogrudan ilgili kullanicinin siparis dokumanina yazildigi icin musterinin `Siparislerim` ekrani da canli olarak guncelleniyor.

Tum siparisleri okumak icin admin repository icinde Firestore collection group sorgusu kullanildi. Ek indeks zorunlulugu dogmamasi icin bu sorguda `orderBy` kullanilmadi; siparisler uygulama tarafinda tarihe gore yeniden eskiye siralandi.

Kitap ekleme akisinda admin kitap adi, yazar, kategori, fiyat, sayfa sayisi, alt baslik ve aciklama gibi alanlari forma giriyor. Form kaydedilince `AdminRepository.addBook()` yeni dokumani `books` koleksiyonuna yaziyor ve kitap listesi tekrar yukleniyor. Bu sayede eklenen kitap magazada hemen gorunuyor.

9. Kayitlarin Saklanmasi ve Verinin Korunmasi

Projenin ilk surumlerinde sepet ve siparis verileri sadece bellekte tutuluyordu. Son surumde bu durum tamamen degisti ve veriler Cloud Firestore uzerinden kaliciya alindi.

- Kitap katalogu `books` koleksiyonundan canli olarak cekiliyor.
- Sepet, `users/{uid}/state/cart` dokumaninda tutuluyor; `snapshots()` ile dinlendigi icin canli senkronize.
- Siparisler, kullanicinin `orders` alt koleksiyonunda her siparis icin ayri bir dokuman olarak tutuluyor. Tarih, toplam tutar, kitap id listesi, durum, kullanici id'si ve kullanici e-postasi burada yer aliyor.
- Firestore guvenlik kurallari sayesinde her kullanici yalnizca kendi verisine erisebiliyor; admin ise sadece gerekli yonetim islemleri icin tum siparisleri okuyup durum guncelleyebiliyor.

Boylece artik uygulama kapansa, telefon yeniden baslatilsa ve hatta baska bir cihazdan giris yapilsa bile sepet ve siparis gecmisi kullanicinin yaninda kaliyor.

10. Kullanici Deneyimi Tarafi

Kullanici deneyiminde sade ama bos olmayan bir yapi kurmaya calistik. Bu deneyimin temeli once Figma tarafinda dusunuldu; yani kullanicinin ilk gorecegi yerlesim, arama alani, filtre butonu ve alt gezinme mantigi daha kod yazilmadan once tasarim olarak kurgulandi. Daha sonra Flutter'a aktarilan bu yapi icinde ana ekranda fazla bolum kullanmak yerine kitap listesini merkeze alan daha temiz bir yaklasim korundu. Ust kisimda sadece arama ve filtre butonu birakildi. Bu sayede kullanici ilk bakista ne yapacagini daha rahat anliyor.

Kitap detay ekranlari da sirf bir kapak ve fiyat gosteren yapida birakilmadi. Bunun yerine kitap ozeti, neden okunmasi gerektigi, kitap bilgileri ve etiketler eklenerek detay sayfasi daha dolu ve daha duzgun hale getirildi.

Siparisler tarafi da yalnizca bir liste olarak birakilmadi. Kullanici gecmis siparisine tikladiginda ayri bir detay sayfasi aciliyor ve siparisteki kitaplari tekrar gorebiliyor. Siparis durum etiketi de musteriye siparisinin hangi asamada oldugunu gosteriyor. Admin tarafinda ise tum siparisleri tek ekranda gormek ve durumu combobox ile degistirmek sunum sirasinda uygulamanin tamamlanmis bir is akisi gibi anlatilmasini sagliyor.

11. Arayuz Tarafi

Arayuz tarafinda klasik beyaz-mor cizgiden uzaklasmak istedik. Bu nedenle once Figma asamasinda mavi-yesil agirlikli bir tasarim dili dusunduk. Arka plan tamamen koyu degil; acik ama soguk bir ton veriyor. Kartlarda beyaz yuzeyler kullanilirken vurgu butonlari ve durum alanlarinda mavi-yesil gecisler tercih edildi. Daha sonra bu tasarim kararlari Flutter tarafinda korunarak gercek arayuze aktarildi.

Bu son duzenlemede kart yapisi da tekrar ele alindi. `BookCard` bileseni ayri dosyaya tasindi ve ic yerlesimi `Expanded` yapisiyla daha dayanikli hale getirildi. Grid tarafinda sabit iki sutun korunurken kart yuksekligi de buna gore ayarlandi. Yeni eklenen `AuthPage` ekrani da uygulamanin geri kalan renk dilini koruyor. Admin ekranlarinda da ayni sade renk dili korundu; siparis durumu etiketleri onay bekliyor, hazirlaniyor, kargoya verildi ve teslim edildi durumlarina gore farkli renklerle gosteriliyor.

12. Kod Bolumleri

Asagida projede rapor acisindan en anlamli gordugumuz dosya ve kod bolumleri yer almaktadir.

12.1. Dosya: pubspec.yaml

Dosyanin gorevi: Flutter projesinin temel bagimliliklarini ve asset tanimlarini belirlemek. Bu son surumde `firebase_core`, `firebase_auth` ve `cloud_firestore` paketleri de buraya eklendi.

Listing 1: pubspec.yaml

```
name: kitap_kiralama
description: "A new Flutter project."
publish_to: 'none'
version: 0.1.0

environment:
  sdk: ^3.10.4

dependencies:
  flutter:
    sdk: flutter
  firebase_core: ^4.9.0
  firebase_auth: ^6.5.1
  cloud_firestore: ^6.4.1

dev_dependencies:
  flutter_test:
    sdk: flutter
  flutter_lints: ^6.0.0

flutter:
  uses-material-design: true
  assets:
    - assets/covers/
```

12.2. Dosya: lib/firebase_options.dart

Dosyanin gorevi: Android, iOS ve Web icin Firebase yapilandirmasini tek yerden saglamak.
Satir sayisi: 74

Bu dosya FlutterFire CLI tarafindan otomatik olarak uretildi. Uygulama baslarken dogru Firebase projesinin acilmasini sagliyor.

12.3. Dosya: lib/app/app_config.dart

Dosyanin gorevi: Admin e-postasi ve siparis durum listesini tek yerde toplamak.
Satir sayisi: 13

Listing 2: AppConfig

```
/// Admin hesabinin e-posta adresi. Bu hesapla giris yapildiginda
/// uygulama normal magaza yerine yonetici panelini acar.
const String kAdminEmail = 'admin@gmail.com';

/// Bir siparisin gecebilecegi durumlar (sirayla).
/// Yeni siparisler ilk durumda ("Onay Bekliyor") baslar; admin
/// bu listeyi izleyerek durumu ileri tasir.
const List<String> kOrderStatuses = <String>[
  'Onay Bekliyor',
  'Hazirlaniyor',
  'Kargoya Verildi',
  'Teslim Edildi',
];
```

Bu dosya sayesinde admin kontrolu ve durum listesi farkli ekranlarda tekrar tekrar yazilmiyor.

12.4. Dosya: lib/main.dart

Dosyanin gorevi: Firebase baslatma, uygulama tema kurulumu ve oturum durumuna gore giris ekrani / admin paneli / normal uygulama secimi.
Satir sayisi: 48

Listing 3: main.dart

```
import 'package:firebase_auth/firebase_auth.dart';
import 'package:firebase_core/firebase_core.dart';
import 'package:flutter/material.dart';
import 'package:kitap_kiralama/app/admin_app.dart';
import 'package:kitap_kiralama/app/app_config.dart';
import 'package:kitap_kiralama/app/book_store_app.dart';
import 'package:kitap_kiralama/firebase_options.dart';
import 'package:kitap_kiralama/screens/auth_page.dart';
import 'package:kitap_kiralama/theme/app_palette.dart';
import 'package:kitap_kiralama/theme/app_theme.dart';

Future<void> main() async {
  WidgetsFlutterBinding.ensureInitialized();
  await Firebase.initializeApp(options: DefaultFirebaseOptions.currentPlatform);
  runApp(const AppRoot());
}

class AppRoot extends StatelessWidget {
  const AppRoot({super.key});

  @override
```

```
  Widget build(BuildContext context) {
    return MaterialApp(
      debugShowCheckedModeBanner: false,
      title: 'Kitap Dukkani',
      theme: buildAppTheme(),
      home: StreamBuilder<User?>(
        stream: FirebaseAuth.instance.authStateChanges(),
        builder: (BuildContext context, AsyncSnapshot<User?> snap) {
          if (snap.connectionState == ConnectionState.waiting) {
            return const Scaffold(
              backgroundColor: AppPalette.background,
              body: Center(child: CircularProgressIndicator()),
            );
          }
          final User? user = snap.data;
          if (user == null) {
            return const AuthPage();
          }
          if (user.email == kAdminEmail) {
            return const AdminApp();
          }
          return const BookStoreApp();
        },
      ),
    );
  }
}
```

`main.dart` artik sadece Flutter calistirma gorevini degil, ayni zamanda uc yonlu oturum kontrolunu de yapiyor. Admin hesabi normal kullanici ekranina dusmeden dogrudan yonetici paneline gidiyor.

12.5. Dosya: lib/screens/auth_page.dart

Dosyanin gorevi: Firebase Authentication uzerinden e-posta sifre ile giris ve kayit isleminin tek bir ekran uzerinden yapilmasini saglamak.
Satir sayisi: 160

Listing 4: AuthPage ekrani

```
import 'package:firebase_auth/firebase_auth.dart';
import 'package:flutter/material.dart';
import 'package:kitap_kiralama/theme/app_palette.dart';

class AuthPage extends StatefulWidget {
  const AuthPage({super.key});
```

```
7
8   @override
9   State<AuthPage> createState() => _AuthPageState();
10 }
11
12 class _AuthPageState extends State<AuthPage> {
13   final TextEditingController _emailController = TextEditingController(
14     text: 'test@gmail.com',
15   );
16   final TextEditingController _passwordController = TextEditingController(
17     text: '12345678',
18   );
19   bool _isLogin = true;
20   bool _loading = false;
21   String? _error;
22
23   @override
24   void dispose() {
25     _emailController.dispose();
26     _passwordController.dispose();
27     super.dispose();
28   }
29
30   Future<void> _submit() async {
31     final String email = _emailController.text.trim();
32     final String password = _passwordController.text;
33     if (email.isEmpty || password.length < 6) {
34       setState(
35         () => _error = 'Gecerli e-posta ve en az 6 karakter sifre girin.',
36       );
37       return;
38     }
39     setState(() {
40       _loading = true;
41       _error = null;
42     });
43     try {
44       if (_isLogin) {
45         await FirebaseAuth.instance.signInWithEmailAndPassword(
46           email: email,
47           password: password,
48         );
49       } else {
50         await FirebaseAuth.instance.createUserWithEmailAndPassword(
51           email: email,
52           password: password,
```

```
        );
      }
    } on FirebaseAuthException catch (e) {
      setState(() => _error = e.message ?? e.code);
    } catch (e) {
      setState(() => _error = e.toString());
    } finally {
      if (mounted) setState(() => _loading = false);
    }
  }

  @override
  Widget build(BuildContext context) {
    return Scaffold(
      backgroundColor: AppPalette.background,
      body: SafeArea(
        child: Center(
          child: SingleChildScrollView(
            padding: const EdgeInsets.all(24),
            child: Column(
              mainAxisSize: MainAxisSize.min,
              crossAxisAlignment: CrossAxisAlignment.stretch,
              children: <Widget>[
                const Icon(
                  Icons.menu_book_rounded,
                  size: 72,
                  color: AppPalette.forest,
                ),
                const SizedBox(height: 12),
                Text(
                  'Kitap Dukkani',
                  textAlign: TextAlign.center,
                  style: Theme.of(context).textTheme.headlineSmall?.copyWith(
                    color: AppPalette.ink,
                    fontWeight: FontWeight.w700,
                  ),
                ),
                const SizedBox(height: 8),
                Text(
                  _isLogin ? 'Hesabina giris yap' : 'Yeni hesap olustur',
                  textAlign: TextAlign.center,
                  style: const TextStyle(color: AppPalette.muted),
                ),
                const SizedBox(height: 32),
                TextField(
                  controller: _emailController,
```

```
                  keyboardType: TextInputType.emailAddress,
                  decoration: const InputDecoration(
                    labelText: 'E-posta',
                    prefixIcon: Icon(Icons.email_outlined),
                  ),
                ),
                const SizedBox(height: 16),
                TextField(
                  controller: _passwordController,
                  obscureText: true,
                  decoration: const InputDecoration(
                    labelText: 'Sifre',
                    prefixIcon: Icon(Icons.lock_outline),
                  ),
                ),
                if (_error != null) ...<Widget>[
                  const SizedBox(height: 16),
                  Text(
                    _error!,
                    style: const TextStyle(color: Colors.redAccent),
                  ),
                ],
                const SizedBox(height: 24),
                FilledButton(
                  onPressed: _loading ? null : _submit,
                  style: FilledButton.styleFrom(
                    backgroundColor: AppPalette.forest,
                    padding: const EdgeInsets.symmetric(vertical: 16),
                  ),
                  child: _loading
                      ? const SizedBox(
                          height: 20,
                          width: 20,
                          child: CircularProgressIndicator(
                            strokeWidth: 2,
                            color: Colors.white,
                          ),
                        )
                      : Text(_isLogin ? 'Giris Yap' : 'Kayit Ol'),
                ),
                const SizedBox(height: 12),
                TextButton(
                  onPressed: _loading
                      ? null
                      : () => setState(() {
                          _isLogin = !_isLogin;
```

```
                          _error = null;
                        }),
                  child: Text(
                    _isLogin
                        ? 'Hesabin yok mu? Kayit ol'
                        : 'Hesabin var mi? Giris yap',
                  ),
                ),
              ],
            ),
          ),
        ),
      ),
    );
  }
}
```

Bu ekran tek bir buton ile hem giris hem kayit yapabiliyor. Form alanlari sunum icin test hesabi bilgileriyle onceden doldurulyor.

12.6. Dosya: lib/data/books_repository.dart

Dosyanin gorevi: Firestore icindeki `books` koleksiyonundan kitaplari cekip `Book` modeline donusturmek.
Satir sayisi: 46

Listing 5: BooksRepository

```
import 'package:cloud_firestore/cloud_firestore.dart';
import 'package:flutter/material.dart';
import 'package:kitap_kiralama/models/book.dart';

class BooksRepository {
  BooksRepository({FirebaseFirestore? firestore})
    : _firestore = firestore ?? FirebaseFirestore.instance;

  final FirebaseFirestore _firestore;

  Future<List<Book>> fetchBooks() async {
    final QuerySnapshot<Map<String, dynamic>> snap = await _firestore
        .collection('books')
        .get();
    return snap.docs.map((QueryDocumentSnapshot<Map<String, dynamic>> d) {
      final Map<String, dynamic> m = d.data();
      return Book(
        id: (m['id'] as String?) ?? d.id,
```

```
        title: (m['title'] as String?) ?? '',
        author: (m['author'] as String?) ?? '',
        category: (m['category'] as String?) ?? '',
        subtitle: (m['subtitle'] as String?) ?? '',
        description: (m['description'] as String?) ?? '',
        whyRead: (m['whyRead'] as String?) ?? '',
        readerNote: (m['readerNote'] as String?) ?? '',
        readingTime: (m['readingTime'] as String?) ?? '',
        edition: (m['edition'] as String?) ?? '',
        rating: (m['rating'] as String?) ?? '0.0',
        pages: (m['pages'] as num?)?.toInt() ?? 0,
        price: (m['price'] as num?)?.toInt() ?? 0,
        language: (m['language'] as String?) ?? '',
        publisher: (m['publisher'] as String?) ?? '',
        highlights: ((m['highlights'] as List<dynamic>?) ?? <dynamic>[])
            .map((dynamic e) => e.toString())
            .toList(),
        startColor: Color(
          (m['startColor'] as num?)?.toInt() ?? 0xFF1C7C8C,
        ).withAlpha(0xFF),
        endColor: Color(
          (m['endColor'] as num?)?.toInt() ?? 0xFF155765,
        ).withAlpha(0xFF),
        coverAsset: m['coverAsset'] as String?,
      );
    }).toList();
  }
}
```

Bu dosya artik `demo_books.dart` icindeki sabit listenin yerini aliyor.

12.7. Dosya: lib/data/user_repository.dart

Dosyanin gorevi: Giris yapmis kullanicinin sepetini ve siparis gecmisini Firestore uzerinde okumak ve yazmak.
Satir sayisi: 82

Listing 6: UserRepository

```
import 'package:cloud_firestore/cloud_firestore.dart';
import 'package:firebase_auth/firebase_auth.dart';
import 'package:kitap_kiralama/models/book.dart';
import 'package:kitap_kiralama/models/order_record.dart';

class UserRepository {
  UserRepository({FirebaseFirestore? firestore, FirebaseAuth? auth})
```

```
    : _firestore = firestore ?? FirebaseFirestore.instance,
      _auth = auth ?? FirebaseAuth.instance;

  final FirebaseFirestore _firestore;
  final FirebaseAuth _auth;

  String get _uid {
    final User? user = _auth.currentUser;
    if (user == null) {
      throw StateError('Kullanici oturumu yok');
    }
    return user.uid;
  }

  DocumentReference<Map<String, dynamic>> get _cartDoc =>
      _firestore.collection('users').doc(_uid).collection('state').doc('cart');

  CollectionReference<Map<String, dynamic>> get _ordersCol =>
      _firestore.collection('users').doc(_uid).collection('orders');

  Stream<List<String>> watchCartIds() {
    return _cartDoc.snapshots().map((DocumentSnapshot<Map<String, dynamic>> s) {
      final Map<String, dynamic>? data = s.data();
      if (data == null) return <String>[];
      final List<dynamic> ids =
          (data['bookIds'] as List<dynamic>?) ?? <dynamic>[];
      return ids.map((dynamic e) => e.toString()).toList();
    });
  }

  Future<void> setCart(List<String> bookIds) async {
    await _cartDoc.set(<String, dynamic>{'bookIds': bookIds});
  }

  Stream<List<OrderRecord>> watchOrders(Map<String, Book> booksById) {
    return _ordersCol.orderBy('createdAt', descending: true).snapshots().map((
      QuerySnapshot<Map<String, dynamic>> snap,
    ) {
      return snap.docs.map((QueryDocumentSnapshot<Map<String, dynamic>> d) {
        final Map<String, dynamic> m = d.data();
        final List<String> ids =
            ((m['bookIds'] as List<dynamic>?) ?? <dynamic>[])
                .map((dynamic e) => e.toString())
                .toList();
        final List<Book> items = <Book>[
          for (final String id in ids)
```

```
            if (booksById[id] != null) booksById[id]!,
        ];
        final Timestamp? ts = m['createdAt'] as Timestamp?;
        return OrderRecord(
          orderCode: (m['orderCode'] as String?) ?? d.id,
          items: items,
          totalPrice: (m['totalPrice'] as num?)?.toInt() ?? 0,
          createdAt: ts?.toDate() ?? DateTime.now(),
          status: (m['status'] as String?) ?? 'Hazirlaniyor',
          userId: (m['userId'] as String?) ?? '',
          userEmail: (m['userEmail'] as String?) ?? '',
        );
      }).toList();
    });
  }

  Future<void> placeOrder(OrderRecord order) async {
    await _ordersCol.doc(order.orderCode).set(<String, dynamic>{
      'orderCode': order.orderCode,
      'bookIds': order.items.map((Book b) => b.id).toList(),
      'totalPrice': order.totalPrice,
      'createdAt': Timestamp.fromDate(order.createdAt),
      'status': 'Onay Bekliyor',
      'userId': _uid,
      'userEmail': _auth.currentUser?.email ?? '',
    });
    await setCart(<String>[]);
  }
}
```

Bu repository sepet icin tek bir dokuman, siparisler icin ise her siparis icin ayri bir dokuman tutar. `placeOrder()` artik her yeni siparise `Onay Bekliyor` durumunu, kullanici id'sini ve kullanici e-postasini da yazar. `snapshots()` kullandigi icin ekranda canli olarak guncellenir.

12.8. Dosya: lib/data/admin_repository.dart

Dosyanin gorevi: Admin panelinin tum siparisleri okumasini, siparis durumunu guncellemesini ve yeni kitap eklemesini saglamak.
Satir sayisi: 68

Listing 7: AdminRepository

```
import 'package:cloud_firestore/cloud_firestore.dart';
import 'package:kitap_kiralama/models/book.dart';
import 'package:kitap_kiralama/models/order_record.dart';

```

```
/// Yonetici islemlerini yurutur: tum kullanicilarin siparislerini okur
/// ve siparis durumlarini gunceller. Sadece admin hesabi tarafindan
/// kullanilir (Firestore kurallari erisimi admin ile sinirlar).
class AdminRepository {
  AdminRepository({FirebaseFirestore? firestore})
    : _firestore = firestore ?? FirebaseFirestore.instance;

  final FirebaseFirestore _firestore;

  /// Tum kullanicilarin "orders" alt koleksiyonlarini tek seferde dinler
  /// (collectionGroup). Siralama, ek bir Firestore indeksi gerektirmemek
  /// icin uygulama tarafinda (yeniden eskiye) yapilir.
  Stream<List<OrderRecord>> watchAllOrders(Map<String, Book> booksById) {
    return _firestore.collectionGroup('orders').snapshots().map((
      QuerySnapshot<Map<String, dynamic>> snap,
    ) {
      final List<OrderRecord> orders = snap.docs.map((
        QueryDocumentSnapshot<Map<String, dynamic>> d,
      ) {
        final Map<String, dynamic> m = d.data();
        final List<String> ids =
            ((m['bookIds'] as List<dynamic>?) ?? <dynamic>[])
                .map((dynamic e) => e.toString())
                .toList();
        final List<Book> items = <Book>[
          for (final String id in ids)
            if (booksById[id] != null) booksById[id]!,
        ];
        final Timestamp? ts = m['createdAt'] as Timestamp?;
        return OrderRecord(
          orderCode: (m['orderCode'] as String?) ?? d.id,
          items: items,
          totalPrice: (m['totalPrice'] as num?)?.toInt() ?? 0,
          createdAt: ts?.toDate() ?? DateTime.now(),
          status: (m['status'] as String?) ?? 'Hazirlaniyor',
          userId: (m['userId'] as String?) ?? '',
          userEmail: (m['userEmail'] as String?) ?? '',
        );
      }).toList();
      orders.sort(
        (OrderRecord a, OrderRecord b) => b.createdAt.compareTo(a.createdAt),
      );
      return orders;
    });
  }

```

```
  /// Bir siparisin durumunu gunceller. Siparis dokumani ilgili
  /// kullanicinin alt koleksiyonunda durdugu icin userId gereklidir.
  Future<void> updateStatus(OrderRecord order, String status) async {
    await _firestore
        .collection('users')
        .doc(order.userId)
        .collection('orders')
        .doc(order.orderCode)
        .update(<String, dynamic>{'status': status});
  }

  /// "books" koleksiyonuna yeni bir kitap ekler. Dokuman id'si data['id']
  /// alanindan alinir.
  Future<void> addBook(Map<String, dynamic> data) async {
    final String id = data['id'] as String;
    await _firestore.collection('books').doc(id).set(data);
  }
}
```

Burada `collectionGroup('orders')` ile tum kullanicilarin siparisleri tek akista dinleniyor. Siralama Firestore yerine uygulama tarafinda yapildigi icin ek indeks zorunlulugu olusmuyor.

12.9. Dosya: lib/app/admin_app.dart

Dosyanin gorevi: Admin hesabiyla giris yapildiginda acilan uc sekmeli yonetici panelini kurmak.
Satir sayisi: 338

Listing 8: AdminApp

```
import 'dart:async';

import 'package:firebase_auth/firebase_auth.dart';
import 'package:flutter/material.dart';
import 'package:kitap_kiralama/data/admin_repository.dart';
import 'package:kitap_kiralama/data/books_repository.dart';
import 'package:kitap_kiralama/models/book.dart';
import 'package:kitap_kiralama/models/order_record.dart';
import 'package:kitap_kiralama/screens/admin_add_book_page.dart';
import 'package:kitap_kiralama/screens/admin_order_detail_page.dart';
import 'package:kitap_kiralama/screens/book_detail_page.dart';
import 'package:kitap_kiralama/screens/store_home_tab.dart';
import 'package:kitap_kiralama/theme/app_palette.dart';
import 'package:kitap_kiralama/widgets/order_status_pill.dart';
import 'package:kitap_kiralama/widgets/summary_row.dart';

```

```
/// Admin hesabiyla giris yapildiginda gosterilen yonetici paneli.
/// Uc sekme icerir: tum siparisler, magaza onizleme ve yeni kitap ekleme.
class AdminApp extends StatefulWidget {
  const AdminApp({super.key});

  @override
  State<AdminApp> createState() => _AdminAppState();
}

class _AdminAppState extends State<AdminApp> {
  final BooksRepository _booksRepo = BooksRepository();
  final AdminRepository _adminRepo = AdminRepository();

  int _selectedIndex = 0;
  List<Book> _books = <Book>[];
  Map<String, Book> _booksById = <String, Book>{};
  List<OrderRecord> _orders = <OrderRecord>[];
  bool _loading = true;
  String? _loadError;

  StreamSubscription<List<OrderRecord>>? _ordersSub;

  @override
  void initState() {
    super.initState();
    _bootstrap();
  }

  @override
  void dispose() {
    _ordersSub?.cancel();
    super.dispose();
  }

  Future<void> _bootstrap() async {
    setState(() {
      _loading = true;
      _loadError = null;
    });
    try {
      await _loadBooks();
      _ordersSub = _adminRepo
          .watchAllOrders(_booksById)
          .listen(
            (List<OrderRecord> orders) {
              if (!mounted) return;
```

```
            setState(() {
              _orders = orders;
              _loading = false;
            });
          },
          onError: (Object e) {
            if (!mounted) return;
            setState(() {
              _loadError = e.toString();
              _loading = false;
            });
          },
        );
  } catch (e) {
    if (!mounted) return;
    setState(() {
      _loadError = e.toString();
      _loading = false;
    });
  }
}

Future<void> _loadBooks() async {
  final List<Book> books = await _booksRepo.fetchBooks();
  if (!mounted) return;
  setState(() {
    _books = books;
    _booksById = <String, Book>{for (final Book b in books) b.id: b};
  });
}

Future<void> _changeStatus(OrderRecord order, String status) {
  return _adminRepo.updateStatus(order, status);
}

Future<void> _addBook(Map<String, dynamic> data) async {
  await _adminRepo.addBook(data);
  await _loadBooks();
}

void _openOrderDetail(OrderRecord order) {
  Navigator.of(context).push(
    MaterialPageRoute<void>(
      builder: (_) =>
          AdminOrderDetailPage(order: order, onStatusChanged: _changeStatus),
    ),
```

```
    );
  }

  void _openBookPreview(Book book) {
    Navigator.of(context).push(
      MaterialPageRoute<void>(
        builder: (_) => BookDetailPage(book: book, previewMode: true),
      ),
    );
  }

  @override
  Widget build(BuildContext context) {
    final List<String> titles = <String>[
      'Yonetici - Siparisler',
      'Yonetici - Magaza',
      'Yonetici - Kitap Ekle',
    ];

    return Scaffold(
      backgroundColor: AppPalette.background,
      appBar: AppBar(
        backgroundColor: AppPalette.surface,
        elevation: 0,
        title: Text(
          titles[_selectedIndex],
          style: const TextStyle(color: AppPalette.ink),
        ),
        actions: <Widget>[
          IconButton(
            icon: const Icon(Icons.logout, color: AppPalette.ink),
            tooltip: 'Cikis',
            onPressed: () => FirebaseAuth.instance.signOut(),
          ),
        ],
      ),
      body: SafeArea(
        child: _loading
            ? const Center(child: CircularProgressIndicator())
            : _loadError != null
            ? _ErrorView(message: _loadError!, onRetry: _bootstrap)
            : IndexedStack(
                index: _selectedIndex,
                children: <Widget>[
                  _OrdersTab(orders: _orders, onOpen: _openOrderDetail),
                  StoreHomeTab(books: _books, onBookSelected: _openBookPreview),
```

```
                  AdminAddBookPage(onAdd: _addBook),
                ],
              ),
      ),
      bottomNavigationBar: NavigationBar(
        selectedIndex: _selectedIndex,
        onDestinationSelected: (int index) =>
            setState(() => _selectedIndex = index),
        height: 74,
        backgroundColor: AppPalette.surface,
        indicatorColor: AppPalette.accentSoft,
        destinations: const <Widget>[
          NavigationDestination(
            icon: Icon(Icons.receipt_long_rounded),
            label: 'Siparisler',
          ),
          NavigationDestination(
            icon: Icon(Icons.storefront_rounded),
            label: 'Magaza',
          ),
          NavigationDestination(
            icon: Icon(Icons.add_box_rounded),
            label: 'Kitap Ekle',
          ),
        ],
      ),
    );
  }
}

class _ErrorView extends StatelessWidget {
  const _ErrorView({required this.message, required this.onRetry});

  final String message;
  final VoidCallback onRetry;

  @override
  Widget build(BuildContext context) {
    return Center(
      child: Padding(
        padding: const EdgeInsets.all(24),
        child: Column(
          mainAxisSize: MainAxisSize.min,
          children: <Widget>[
            Text(
              'Siparisler yuklenemedi:\n$message',
```

```
              textAlign: TextAlign.center,
            ),
            const SizedBox(height: 16),
            FilledButton(onPressed: onRetry, child: const Text('Tekrar dene')),
          ],
        ),
      ),
    );
  }
}

class _OrdersTab extends StatelessWidget {
  const _OrdersTab({required this.orders, required this.onOpen});

  final List<OrderRecord> orders;
  final void Function(OrderRecord order) onOpen;

  @override
  Widget build(BuildContext context) {
    final TextTheme textTheme = Theme.of(context).textTheme;

    if (orders.isEmpty) {
      return Center(
        child: Text('Henuz siparis yok', style: textTheme.titleLarge),
      );
    }

    return Padding(
      padding: const EdgeInsets.fromLTRB(16, 16, 16, 16),
      child: Column(
        crossAxisAlignment: CrossAxisAlignment.start,
        children: <Widget>[
          Text('Tum Siparisler', style: textTheme.headlineSmall),
          const SizedBox(height: 6),
          Text(
            '${orders.length} siparis bulunuyor',
            style: textTheme.bodyMedium,
          ),
          const SizedBox(height: 16),
          Expanded(
            child: ListView.separated(
              itemCount: orders.length,
              separatorBuilder: (BuildContext context, int index) =>
                  const SizedBox(height: 14),
              itemBuilder: (BuildContext context, int index) {
                final OrderRecord order = orders[index];
```

```
                return Material(
                  color: Colors.transparent,
                  child: InkWell(
                    borderRadius: BorderRadius.circular(26),
                    onTap: () => onOpen(order),
                    child: Ink(
                      padding: const EdgeInsets.all(18),
                      decoration: BoxDecoration(
                        color: Colors.white,
                        borderRadius: BorderRadius.circular(26),
                      ),
                      child: Column(
                        crossAxisAlignment: CrossAxisAlignment.start,
                        children: <Widget>[
                          Row(
                            children: <Widget>[
                              Expanded(
                                child: Column(
                                  crossAxisAlignment: CrossAxisAlignment.start,
                                  children: <Widget>[
                                    Text(
                                      order.orderCode,
                                      style: textTheme.titleMedium?.copyWith(
                                        color: AppPalette.forest,
                                      ),
                                    ),
                                    const SizedBox(height: 4),
                                    Text(
                                      order.createdLabel,
                                      style: textTheme.bodyMedium,
                                    ),
                                  ],
                                ),
                              ),
                              OrderStatusPill(label: order.statusLabel),
                            ],
                          ),
                          const SizedBox(height: 12),
                          Row(
                            children: <Widget>[
                              const Icon(
                                Icons.person_outline,
                                size: 18,
                                color: AppPalette.muted,
                              ),
                              const SizedBox(width: 6),
```

```
                                Expanded(
                                  child: Text(
                                    order.userEmail.isEmpty
                                        ? 'Bilinmeyen kullanici'
                                        : order.userEmail,
                                    style: textTheme.bodyLarge,
                                  ),
                                ),
                                const Icon(
                                  Icons.chevron_right_rounded,
                                  color: AppPalette.muted,
                                ),
                              ],
                            ),
                            const SizedBox(height: 14),
                            Row(
                              children: <Widget>[
                                Expanded(
                                  child: SummaryRow(
                                    label: 'Kitap Sayisi',
                                    value: '${order.items.length}',
                                  ),
                                ),
                                const SizedBox(width: 14),
                                Expanded(
                                  child: SummaryRow(
                                    label: 'Toplam',
                                    value: '${order.totalPrice} TL',
                                    strong: true,
                                  ),
                                ),
                              ],
                            ),
                          ],
                        ),
                      ),
                    ),
                  );
                },
              ),
            ),
          ],
        ),
      );
  }
}
```

`AdminApp` icinde `Siparisler`, `Magaza` ve `Kitap Ekle` sekmeleri bulunuyor. Magaza sekmesi admin icin onizleme olarak calisiyor; kitap detayinda satin alma cubugu gosterilmiyor.

12.10. Dosya: lib/screens/admin_order_detail_page.dart

Dosyanin gorevi: Adminin bir siparisin kitaplarini, musteri bilgisini ve kargo durumunu ayrintili gormesini saglamak.
Satir sayisi: 287

Listing 9: Admin siparis detayi

```
import 'package:flutter/material.dart';
import 'package:kitap_kiralama/app/app_config.dart';
import 'package:kitap_kiralama/models/book.dart';
import 'package:kitap_kiralama/models/order_record.dart';
import 'package:kitap_kiralama/theme/app_palette.dart';
import 'package:kitap_kiralama/widgets/book_cover.dart';
import 'package:kitap_kiralama/widgets/detail_info_tile.dart';
import 'package:kitap_kiralama/widgets/order_status_pill.dart';

/// Admin'in tek bir siparisin tum detaylarini gordugu ve kargo durumunu
/// bir acilir liste (combobox) ile degistirdigi ekran.
class AdminOrderDetailPage extends StatefulWidget {
  const AdminOrderDetailPage({
    super.key,
    required this.order,
    required this.onStatusChanged,
  });

  final OrderRecord order;

  /// Yeni durum secildiginde cagrilir; Firestore guncellemesini yapar.
  final Future<void> Function(OrderRecord order, String status) onStatusChanged;

  @override
  State<AdminOrderDetailPage> createState() => _AdminOrderDetailPageState();
}

class _AdminOrderDetailPageState extends State<AdminOrderDetailPage> {
  late String _status;
  bool _saving = false;

  @override
  void initState() {
    super.initState();
```

```
    _status = widget.order.status;
  }

  Future<void> _onChanged(String? value) async {
    if (value == null || value == _status) return;
    setState(() => _saving = true);
    try {
      await widget.onStatusChanged(widget.order, value);
      if (!mounted) return;
      setState(() {
        _status = value;
        _saving = false;
      });
      ScaffoldMessenger.of(
        context,
      ).showSnackBar(SnackBar(content: Text('Durum guncellendi: $value')));
    } catch (e) {
      if (!mounted) return;
      setState(() => _saving = false);
      ScaffoldMessenger.of(
        context,
      ).showSnackBar(SnackBar(content: Text('Guncellenemedi: $e')));
    }
  }

  @override
  Widget build(BuildContext context) {
    final TextTheme textTheme = Theme.of(context).textTheme;
    final OrderRecord order = widget.order;

    return Scaffold(
      backgroundColor: AppPalette.background,
      appBar: AppBar(
        backgroundColor: AppPalette.background,
        surfaceTintColor: Colors.transparent,
        title: const Text('Siparis Detayi'),
      ),
      body: ListView(
        padding: const EdgeInsets.fromLTRB(20, 12, 20, 24),
        children: <Widget>[
          Container(
            padding: const EdgeInsets.all(22),
            decoration: BoxDecoration(
              borderRadius: BorderRadius.circular(30),
              gradient: const LinearGradient(
                colors: <Color>[
```

```
                AppPalette.deepForest,
                AppPalette.forest,
                AppPalette.accent,
              ],
              begin: Alignment.topLeft,
              end: Alignment.bottomRight,
            ),
          ),
          child: Column(
            crossAxisAlignment: CrossAxisAlignment.start,
            children: <Widget>[
              OrderStatusPill(label: _status, inverse: true),
              const SizedBox(height: 18),
              Text(
                order.orderCode,
                style: textTheme.headlineSmall?.copyWith(color: Colors.white),
              ),
              const SizedBox(height: 8),
              Text(
                'Siparis ${order.createdLabel} tarihinde olusturuldu.',
                style: textTheme.bodyLarge?.copyWith(
                  color: Colors.white.withValues(alpha: 0.86),
                ),
              ),
              const SizedBox(height: 18),
              Row(
                children: <Widget>[
                  Expanded(
                    child: DetailInfoTile(
                      label: 'Kitap',
                      value: '${order.items.length} adet',
                      dark: true,
                    ),
                  ),
                  const SizedBox(width: 12),
                  Expanded(
                    child: DetailInfoTile(
                      label: 'Toplam',
                      value: '${order.totalPrice} TL',
                      dark: true,
                    ),
                  ),
                ],
              ),
            ],
          ),
```

```
          ),
          const SizedBox(height: 20),
          Container(
            padding: const EdgeInsets.all(20),
            decoration: BoxDecoration(
              color: Colors.white,
              borderRadius: BorderRadius.circular(28),
            ),
            child: Column(
              crossAxisAlignment: CrossAxisAlignment.start,
              children: <Widget>[
                Text('Musteri', style: textTheme.titleLarge),
                const SizedBox(height: 12),
                Row(
                  children: <Widget>[
                    const Icon(
                      Icons.person_outline,
                      size: 20,
                      color: AppPalette.forest,
                    ),
                    const SizedBox(width: 8),
                    Expanded(
                      child: Text(
                        order.userEmail.isEmpty
                            ? 'Bilinmeyen kullanici'
                            : order.userEmail,
                        style: textTheme.bodyLarge,
                      ),
                    ),
                  ],
                ),
              ],
            ),
          ),
          const SizedBox(height: 20),
          Container(
            padding: const EdgeInsets.all(20),
            decoration: BoxDecoration(
              color: Colors.white,
              borderRadius: BorderRadius.circular(28),
            ),
            child: Column(
              crossAxisAlignment: CrossAxisAlignment.start,
              children: <Widget>[
                Text('Kargo Durumu', style: textTheme.titleLarge),
                const SizedBox(height: 6),
```

```
              Text(
                'Durumu degistirin; musterinin siparisi aninda guncellenir.',
                style: textTheme.bodyMedium,
              ),
              const SizedBox(height: 14),
              Container(
                padding: const EdgeInsets.symmetric(horizontal: 16),
                decoration: BoxDecoration(
                  color: AppPalette.cardSoft,
                  borderRadius: BorderRadius.circular(16),
                ),
                child: Row(
                  children: <Widget>[
                    Expanded(
                      child: DropdownButtonHideUnderline(
                        child: DropdownButton<String>(
                          value: _status,
                          isExpanded: true,
                          borderRadius: BorderRadius.circular(16),
                          items: kOrderStatuses
                              .map(
                                (String s) => DropdownMenuItem<String>(
                                  value: s,
                                  child: Text(s),
                                ),
                              )
                              .toList(),
                          onChanged: _saving ? null : _onChanged,
                        ),
                      ),
                    ),
                    if (_saving)
                      const SizedBox(
                        height: 18,
                        width: 18,
                        child: CircularProgressIndicator(strokeWidth: 2),
                      ),
                  ],
                ),
              ),
            ],
          ),
        ),
        const SizedBox(height: 20),
        Container(
          padding: const EdgeInsets.all(20),
```

```
            decoration: BoxDecoration(
              color: Colors.white,
              borderRadius: BorderRadius.circular(28),
            ),
            child: Column(
              crossAxisAlignment: CrossAxisAlignment.start,
              children: <Widget>[
                Text('Siparisteki Kitaplar', style: textTheme.titleLarge),
                const SizedBox(height: 16),
                if (order.items.isEmpty)
                  Text('Kitap bilgisi yok', style: textTheme.bodyMedium)
                else
                  ...order.items.map((Book item) {
                    return Padding(
                      padding: const EdgeInsets.only(bottom: 14),
                      child: Container(
                        padding: const EdgeInsets.all(12),
                        decoration: BoxDecoration(
                          color: AppPalette.cardSoft.withValues(alpha: 0.45),
                          borderRadius: BorderRadius.circular(22),
                        ),
                        child: Row(
                          children: <Widget>[
                            SizedBox(
                              width: 72,
                              child: BookCover(
                                book: item,
                                height: 104,
                                borderRadius: 18,
                                compact: true,
                              ),
                            ),
                            const SizedBox(width: 14),
                            Expanded(
                              child: Column(
                                crossAxisAlignment: CrossAxisAlignment.start,
                                children: <Widget>[
                                  Text(
                                    item.title,
                                    style: textTheme.titleMedium,
                                  ),
                                  const SizedBox(height: 4),
                                  Text(
                                    item.author,
                                    style: textTheme.bodyMedium,
                                  ),
```

```
                                    const SizedBox(height: 8),
                                    Text(
                                      item.priceLabel,
                                      style: textTheme.titleMedium?.copyWith(
                                        color: AppPalette.forest,
                                      ),
                                    ),
                                  ],
                                ),
                              ),
                            ],
                          ),
                        ),
                      );
                    }),
                ],
              ),
            ),
          ],
        ),
      );
  }
}
```

Bu ekranda durum combobox ile degistiriliyor. Degisiklik kaydedilince Firestore'daki siparis dokumani guncelleniyor.

12.11. Dosya: lib/screens/admin_add_book_page.dart

Dosyanin gorevi: Adminin yeni kitap bilgilerini form uzerinden girip `books` koleksiyonuna eklemesini saglamak.
Satir sayisi: 190

Listing 10: Admin kitap ekleme formu

```
import 'package:flutter/material.dart';
import 'package:kitap_kiralama/theme/app_palette.dart';

/// Admin'in yeni kitap eklemesini saglayan form. Temel alanlari ister,
/// kalan alanlara makul varsayilanlar verir ve "books" koleksiyonuna yazar.
class AdminAddBookPage extends StatefulWidget {
  const AdminAddBookPage({super.key, required this.onAdd});

  /// Form gonderildiginde kitap verisini Firestore'a yazar.
  final Future<void> Function(Map<String, dynamic> data) onAdd;

```

```
  @override
  State<AdminAddBookPage> createState() => _AdminAddBookPageState();
}

class _AdminAddBookPageState extends State<AdminAddBookPage> {
  final GlobalKey<FormState> _formKey = GlobalKey<FormState>();

  final TextEditingController _title = TextEditingController();
  final TextEditingController _author = TextEditingController();
  final TextEditingController _category = TextEditingController();
  final TextEditingController _price = TextEditingController();
  final TextEditingController _pages = TextEditingController();
  final TextEditingController _subtitle = TextEditingController();
  final TextEditingController _description = TextEditingController();

  bool _saving = false;

  @override
  void dispose() {
    _title.dispose();
    _author.dispose();
    _category.dispose();
    _price.dispose();
    _pages.dispose();
    _subtitle.dispose();
    _description.dispose();
    super.dispose();
  }

  String _slug(String input) {
    final String base = input
        .toLowerCase()
        .replaceAll(RegExp(r'[^a-z0-9]+'), '-')
        .replaceAll(RegExp(r'^-+|-+$'), '');
    final String suffix = DateTime.now().millisecondsSinceEpoch
        .toString()
        .substring(8);
    return base.isEmpty ? 'kitap-$suffix' : '$base-$suffix';
  }

  Future<void> _submit() async {
    if (!(_formKey.currentState?.validate() ?? false)) return;
    setState(() => _saving = true);

    final String title = _title.text.trim();
    final String id = _slug(title);
```

```
    final Map<String, dynamic> data = <String, dynamic>{
      'id': id,
      'title': title,
      'author': _author.text.trim(),
      'category': _category.text.trim().isEmpty
          ? 'Genel'
          : _category.text.trim(),
      'subtitle': _subtitle.text.trim(),
      'description': _description.text.trim(),
      'whyRead': '',
      'readerNote': '',
      'readingTime': '5 saat',
      'edition': '1. Baski',
      'rating': '4.5',
      'pages': int.tryParse(_pages.text.trim()) ?? 0,
      'price': int.tryParse(_price.text.trim()) ?? 0,
      'language': 'Turkce',
      'publisher': '',
      'highlights': <String>[],
      'startColor': 0xFF1C7C8C,
      'endColor': 0xFF155765,
    };

    try {
      await widget.onAdd(data);
      if (!mounted) return;
      setState(() => _saving = false);
      _formKey.currentState?.reset();
      for (final TextEditingController c in <TextEditingController>[
        _title,
        _author,
        _category,
        _price,
        _pages,
        _subtitle,
        _description,
      ]) {
        c.clear();
      }
      ScaffoldMessenger.of(
        context,
      ).showSnackBar(SnackBar(content: Text('"$title" eklendi.')));
    } catch (e) {
      if (!mounted) return;
      setState(() => _saving = false);
      ScaffoldMessenger.of(
```

```
104         context,
105       ).showSnackBar(SnackBar(content: Text('Eklenemedi: $e')));
106     }
107   }
108
109   @override
110   Widget build(BuildContext context) {
111     final TextTheme textTheme = Theme.of(context).textTheme;
112     return SafeArea(
113       child: Form(
114         key: _formKey,
115         child: ListView(
116           padding: const EdgeInsets.fromLTRB(20, 18, 20, 24),
117           children: <Widget>[
118             Text('Yeni Kitap Ekle', style: textTheme.headlineSmall),
119             const SizedBox(height: 6),
120             Text(
121               'Eklenen kitap aninda magazada gorunur.',
122               style: textTheme.bodyMedium,
123             ),
124             const SizedBox(height: 20),
125             _field(_title, 'Kitap adi', required: true),
126             _field(_author, 'Yazar', required: true),
127             _field(_category, 'Kategori (orn. Roman)'),
128             _field(
129               _price,
130               'Fiyat (TL)',
131               keyboardType: TextInputType.number,
132               required: true,
133             ),
134             _field(_pages, 'Sayfa sayisi', keyboardType: TextInputType.number),
135             _field(_subtitle, 'Alt baslik'),
136             _field(_description, 'Aciklama', maxLines: 4),
137             const SizedBox(height: 8),
138             FilledButton(
139               onPressed: _saving ? null : _submit,
140               style: FilledButton.styleFrom(
141                 backgroundColor: AppPalette.forest,
142                 padding: const EdgeInsets.symmetric(vertical: 16),
143               ),
144               child: _saving
145                   ? const SizedBox(
146                       height: 20,
147                       width: 20,
148                       child: CircularProgressIndicator(
149                         strokeWidth: 2,
```

```
                        color: Colors.white,
                      ),
                    )
                  : const Text('Kitabi Ekle'),
            ),
          ],
        ),
      ),
    );
  }

  Widget _field(
    TextEditingController controller,
    String label, {
    bool required = false,
    TextInputType keyboardType = TextInputType.text,
    int maxLines = 1,
  }) {
    return Padding(
      padding: const EdgeInsets.only(bottom: 14),
      child: TextFormField(
        controller: controller,
        keyboardType: keyboardType,
        maxLines: maxLines,
        decoration: InputDecoration(
          labelText: required ? '$label *' : label,
          filled: true,
          fillColor: Colors.white,
          border: OutlineInputBorder(
            borderRadius: BorderRadius.circular(16),
            borderSide: BorderSide.none,
          ),
        ),
        validator: required
            ? (String? v) =>
                  (v == null || v.trim().isEmpty) ? 'Bu alan zorunlu' : null
            : null,
      ),
    );
  }
}
```

Form kaydedildikten sonra kitap Firestore'a yaziliyor ve magaza listesi yeniden yukleniyor.

12.12. Dosya: firestore.rules

Dosyanin gorevi: Firestore tarafinda kimlerin neye erisecegini belirlemek.
Satir sayisi: 25

Listing 11: firestore.rules

```
rules_version = '2';
service cloud.firestore {
  match /databases/{database}/documents {
    function isAdmin() {
      return request.auth != null
        && request.auth.token.email == 'admin@gmail.com';
    }

    match /books/{bookId} {
      allow read: if request.auth != null;
      allow write: if isAdmin();
    }

    match /users/{userId}/{document=**} {
      allow read, write: if request.auth != null && request.auth.uid == userId;
    }

    // Admin tum kullanicilarin siparislerini okuyabilir (collectionGroup)
    // ve siparis durumunu guncelleyebilir.
    match /{path=**}/orders/{orderId} {
      allow read: if isAdmin();
      allow update: if isAdmin();
    }
  }
}
```

Kitap katalogu giris yapmis kullanicilar tarafindan okunabiliyor, kitap yazma yetkisi admin ile sinirli kaliyor. Kullanici verileri sahibine acik; admin ise tum siparisleri okuyup sadece durum guncellemesi yapabiliyor.

12.13. Dosya: lib/app/book_store_app.dart

Dosyanin gorevi: Uygulamanin ana sekmelerini, kitap yuklemesini, sepet ve siparis akisini ve cikis butonunu yonetmek.
Satir sayisi: 229

Listing 12: Uygulama kabugu

```
import 'dart:async';

```

```dart
import 'package:firebase_auth/firebase_auth.dart';
import 'package:flutter/material.dart';
import 'package:kitap_kiralama/data/books_repository.dart';
import 'package:kitap_kiralama/data/user_repository.dart';
import 'package:kitap_kiralama/models/book.dart';
import 'package:kitap_kiralama/models/order_record.dart';
import 'package:kitap_kiralama/screens/book_detail_page.dart';
import 'package:kitap_kiralama/screens/cart_tab.dart';
import 'package:kitap_kiralama/screens/orders_tab.dart';
import 'package:kitap_kiralama/screens/store_home_tab.dart';
import 'package:kitap_kiralama/theme/app_palette.dart';
import 'package:kitap_kiralama/widgets/cart_icon_badge.dart';

class BookStoreApp extends StatefulWidget {
  const BookStoreApp({super.key});

  @override
  State<BookStoreApp> createState() => _BookStoreAppState();
}

class _BookStoreAppState extends State<BookStoreApp> {
  final BooksRepository _booksRepo = BooksRepository();
  final UserRepository _userRepo = UserRepository();

  int _selectedIndex = 0;
  List<Book> _books = <Book>[];
  Map<String, Book> _booksById = <String, Book>{};
  List<Book> _cart = <Book>[];
  List<OrderRecord> _orders = <OrderRecord>[];
  bool _loading = true;
  String? _loadError;

  StreamSubscription<List<String>>? _cartSub;
  StreamSubscription<List<OrderRecord>>? _ordersSub;

  @override
  void initState() {
    super.initState();
    _bootstrap();
  }

  @override
  void dispose() {
    _cartSub?.cancel();
    _ordersSub?.cancel();
    super.dispose();
```

```
  }

  Future<void> _bootstrap() async {
    setState(() {
      _loading = true;
      _loadError = null;
    });
    try {
      final List<Book> books = await _booksRepo.fetchBooks();
      final Map<String, Book> byId = <String, Book>{
        for (final Book b in books) b.id: b,
      };
      if (!mounted) return;
      setState(() {
        _books = books;
        _booksById = byId;
        _loading = false;
      });

      _cartSub = _userRepo.watchCartIds().listen((List<String> ids) {
        if (!mounted) return;
        setState(() {
          _cart = <Book>[
            for (final String id in ids)
              if (_booksById[id] != null) _booksById[id]!,
          ];
        });
      });

      _ordersSub = _userRepo.watchOrders(_booksById).listen((
        List<OrderRecord> orders,
      ) {
        if (!mounted) return;
        setState(() => _orders = orders);
      });
    } catch (e) {
      if (!mounted) return;
      setState(() {
        _loadError = e.toString();
        _loading = false;
      });
    }
  }

  bool _isInCart(Book book) => _cart.any((Book item) => item.id == book.id);

```

```
95  Future<bool> _addToCart(Book book) async {
96    if (_isInCart(book)) return false;
97    final List<String> next = <String>[..._cart.map((Book b) => b.id), book.id];
98    await _userRepo.setCart(next);
99    return true;
100 }
101
102 Future<void> _removeFromCart(Book book) async {
103   final List<String> next = _cart
104       .where((Book b) => b.id != book.id)
105       .map((Book b) => b.id)
106       .toList();
107   await _userRepo.setCart(next);
108 }
109
110 void _openCart() => setState(() => _selectedIndex = 1);
111 void _openBooks() => setState(() => _selectedIndex = 0);
112
113 Future<void> _completeOrder(OrderRecord order) async {
114   await _userRepo.placeOrder(order);
115   if (!mounted) return;
116   setState(() => _selectedIndex = 2);
117 }
118
119 @override
120 Widget build(BuildContext context) {
121   return Builder(
122     builder: (BuildContext appContext) {
123       return Scaffold(
124         appBar: AppBar(
125           backgroundColor: AppPalette.surface,
126           elevation: 0,
127           title: const Text(
128             'Kitap Dukkani',
129             style: TextStyle(color: AppPalette.ink),
130           ),
131           actions: <Widget>[
132             IconButton(
133               icon: const Icon(Icons.logout, color: AppPalette.ink),
134               tooltip: 'Cikis',
135               onPressed: () => FirebaseAuth.instance.signOut(),
136             ),
137           ],
138         ),
139         body: SafeArea(
140           child: _loading
```

```
                ? const Center(child: CircularProgressIndicator())
                : _loadError != null
                ? Center(
                    child: Padding(
                      padding: const EdgeInsets.all(24),
                      child: Column(
                        mainAxisSize: MainAxisSize.min,
                        children: <Widget>[
                          Text(
                            'Veri yuklenemedi:\n$_loadError',
                            textAlign: TextAlign.center,
                          ),
                          const SizedBox(height: 16),
                          FilledButton(
                            onPressed: _bootstrap,
                            child: const Text('Tekrar dene'),
                          ),
                        ],
                      ),
                    ),
                  )
                : IndexedStack(
                    index: _selectedIndex,
                    children: <Widget>[
                      StoreHomeTab(
                        books: _books,
                        onBookSelected: (Book book) {
                          Navigator.of(appContext).push(
                            MaterialPageRoute<void>(
                              builder: (BuildContext detailContext) {
                                return BookDetailPage(
                                  book: book,
                                  isInCart: _isInCart(book),
                                  onAddToCart: () async {
                                    return _addToCart(book);
                                  },
                                  onOpenCart: _openCart,
                                );
                              },
                            ),
                          );
                        },
                      ),
                      CartTab(
                        cartItems: _cart,
                        onRemove: _removeFromCart,
```

```
                        onContinueShopping: _openBooks,
                        onPlaceOrder: _completeOrder,
                      ),
                      OrdersTab(
                        orders: _orders,
                        onContinueShopping: _openBooks,
                      ),
                    ],
                  ),
            ),
            bottomNavigationBar: NavigationBar(
              selectedIndex: _selectedIndex,
              onDestinationSelected: (int index) =>
                  setState(() => _selectedIndex = index),
              height: 74,
              backgroundColor: AppPalette.surface,
              indicatorColor: AppPalette.accentSoft,
              destinations: <Widget>[
                const NavigationDestination(
                  icon: Icon(Icons.grid_view_rounded),
                  selectedIcon: Icon(Icons.grid_view_rounded),
                  label: 'Kitaplar',
                ),
                NavigationDestination(
                  icon: CartIconBadge(count: _cart.length, isCompact: true),
                  selectedIcon: CartIconBadge(
                    count: _cart.length,
                    isCompact: true,
                  ),
                  label: 'Sepet',
                ),
                const NavigationDestination(
                  icon: Icon(Icons.receipt_long_rounded),
                  selectedIcon: Icon(Icons.receipt_long_rounded),
                  label: 'Siparislerim',
                ),
              ],
            ),
          );
        },
      );
    }
}
```

Bu kabuk artik kitaplari Firestore'dan cekiyor, sepet ve siparisleri repository uzerinden okuyup yaziyor. `AppBar` icine eklenen cikis butonu ile oturum kapatilabiliyor.

12.14. Dosya: lib/screens/cart_tab.dart

Dosyanin gorevi: Sepetteki urunleri listelemek, toplam tutari hesaplamak ve siparisi olusturmak.

Bu ekrandaki `onPlaceOrder` cagrisi eskiden yalnizca bellege ekleme yapiyordu. Yeni surumde bu cagri `BookStoreApp` icinden `UserRepository.placeOrder` uzerinden Firestore'a kayit atiyor, ardindan sepet temizleniyor.

12.15. Dosya: lib/models/order_record.dart

Dosyanin gorevi: Siparis bilgisini ekranlar ve repository'ler arasinda tasimak.
Satir sayisi: 51

Listing 13: OrderRecord modeli

```
import 'package:kitap_kiralama/models/book.dart';

class OrderRecord {
  const OrderRecord({
    required this.orderCode,
    required this.items,
    required this.totalPrice,
    required this.createdAt,
    this.status = 'Onay Bekliyor',
    this.userId = '',
    this.userEmail = '',
  });

  final String orderCode;
  final List<Book> items;
  final int totalPrice;
  final DateTime createdAt;

  /// Siparisin guncel durumu (orn. "Hazirlaniyor", "Kargoya Verildi").
  final String status;

  /// Siparisi veren kullanicinin Firebase UID'i (admin guncellemesi icin).
  final String userId;

  /// Siparisi veren kullanicinin e-posta adresi (admin panelinde gosterilir).
  final String userEmail;

  String get createdLabel {
    final String day = createdAt.day.toString().padLeft(2, '0');
    final String month = createdAt.month.toString().padLeft(2, '0');
    final String year = createdAt.year.toString();
```

```
    final String hour = createdAt.hour.toString().padLeft(2, '0');
    final String minute = createdAt.minute.toString().padLeft(2, '0');
    return '$day.$month.$year • $hour:$minute';
  }

  String get bookTitles {
    return items.map((Book book) => book.title).join(', ');
  }

  String get statusLabel => status;

  String get trackingCode => 'TRK-${orderCode.substring(2)}';

  String get estimatedDeliveryLabel {
    final DateTime estimatedDate = createdAt.add(const Duration(days: 2));
    final String day = estimatedDate.day.toString().padLeft(2, '0');
    final String month = estimatedDate.month.toString().padLeft(2, '0');
    return '$day.$month icinde teslim';
  }
}
```

Model artik siparis durumunu, kullanici id'sini ve kullanici e-postasini da tutuyor. Bu alanlar admin panelinde siparisi dogru kullaniciyle eslestirmek icin kullaniliyor.

13. Sonuc ve Genel Degerlendirme

Bu projede hedefimiz asiri buyuk bir mimari kurmak degildi. Bunun yerine final projesi seviyesinde derli toplu, duzgun gorunen ve adim adim anlatilabilecek bir kitap uygulamasi olusturduk. Geldigimiz noktada uygulama:

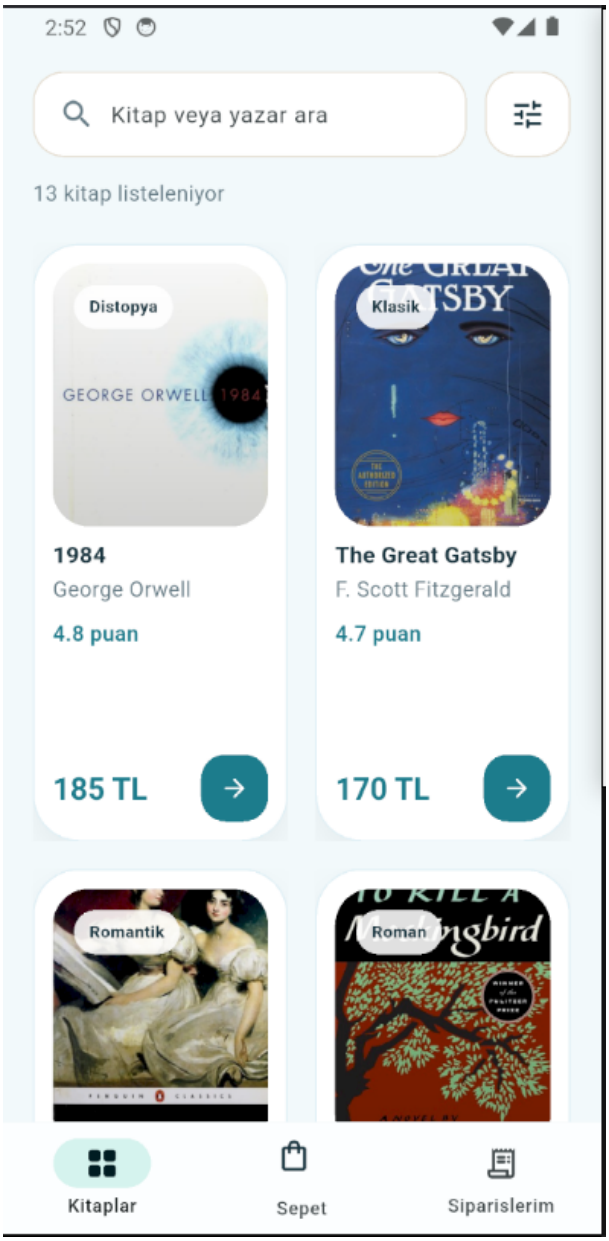
- e-posta sifre ile giris ve kayit yapiyor,
- kitaplari Firestore uzerinden listeliyor,
- arama ve filtreleme yapiyor,
- detay sayfasi aciyor,
- sepete ekliyor ve sepeti Firestore'da saklayip canli senkronize ediyor,
- siparis verdiriyor ve siparisleri kullanici bazli kaydediyor,
- siparisleri ayri ekranda tutuyor,
- yonetici hesabiyla tum siparisleri tek ekranda yonetiyor,

- kargo durumunu onay bekliyor, hazirlaniyor, kargoya verildi ve teslim edildi adimlari arasinda degistirebiliyor,
- yeni kitap ekleyebiliyor,
- admin magazayi satin alma cubugu olmadan onizleyebiliyor,
- guvenlik kurallari sayesinde kullanici verisini sadece sahibine, yonetim islemlerini de admin hesabina aciyor.

Bu son duzenleme ile birlikte proje yalnizca islevsel degil, kod yapisi ve veri katmani acisindan da daha savunulabilir hale geldi. `main.dart` sade bir giris noktasina indirildi; ana akis, admin paneli, ekranlar, modeller, repository'ler, ortak widgetlar ve Firebase yapilandirmasi kendi dosyalarina ayrildi. Bu haliyle proje, hem arayuz hem temel akis hem kod organizasyonu hem de bulut veri katmani acisindan tamamlanmis bir final odevi niteligindedir.

14. Ekran Goruntuleri

Asagidaki gorseller uygulamanin farkli ekranlarindan alinmistir.



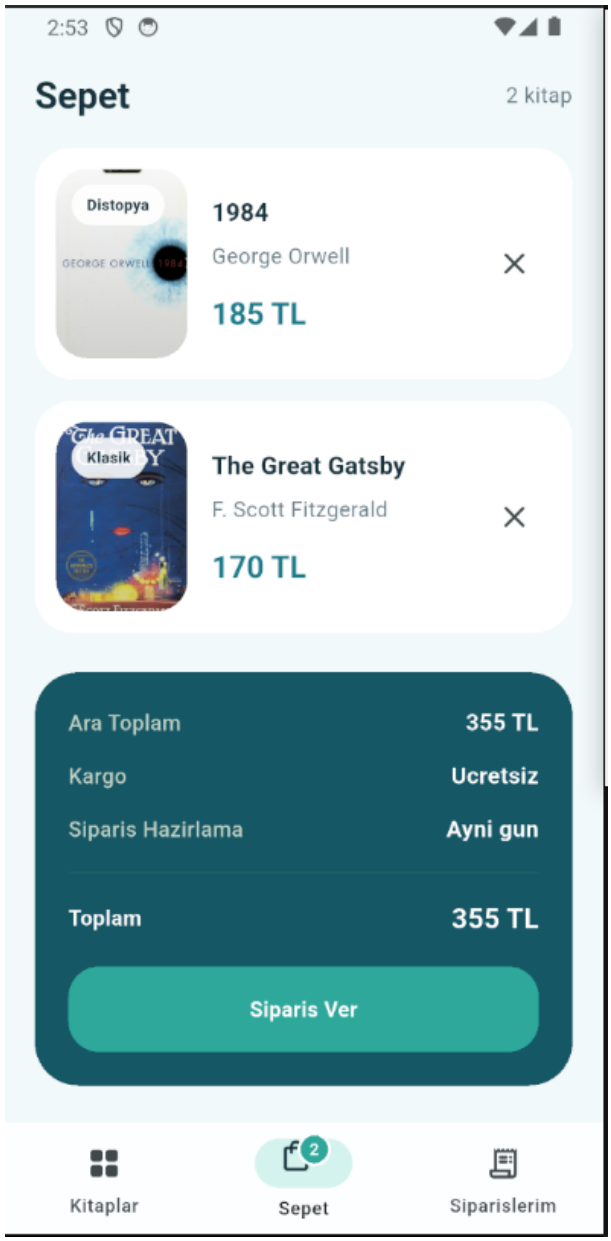
(a) Ekran goruntusu 1



(b) Ekran goruntusu 2



(a) Ekran goruntusu 3



(b) Ekran goruntusu 4

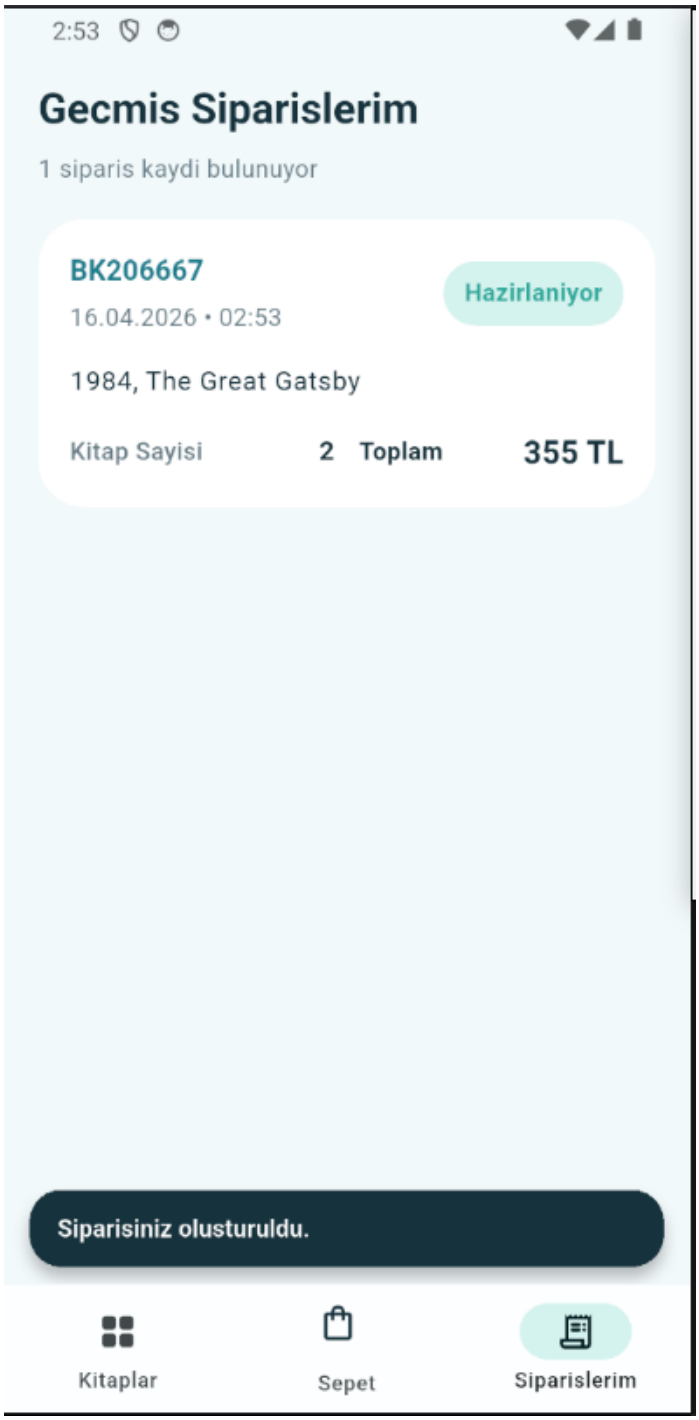
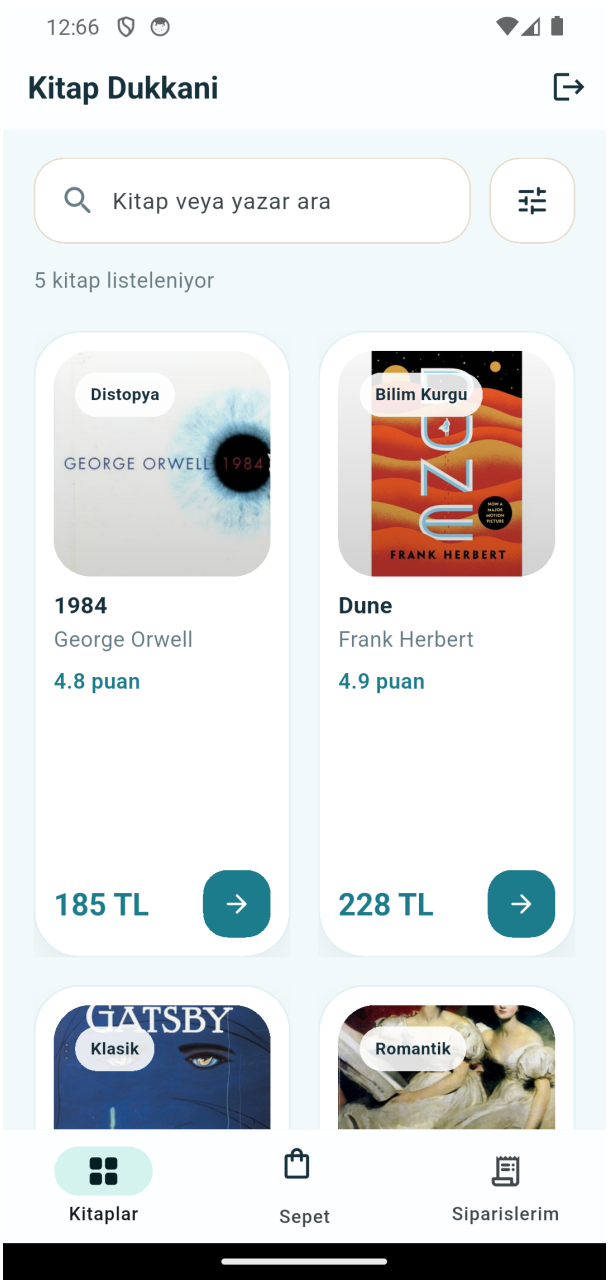
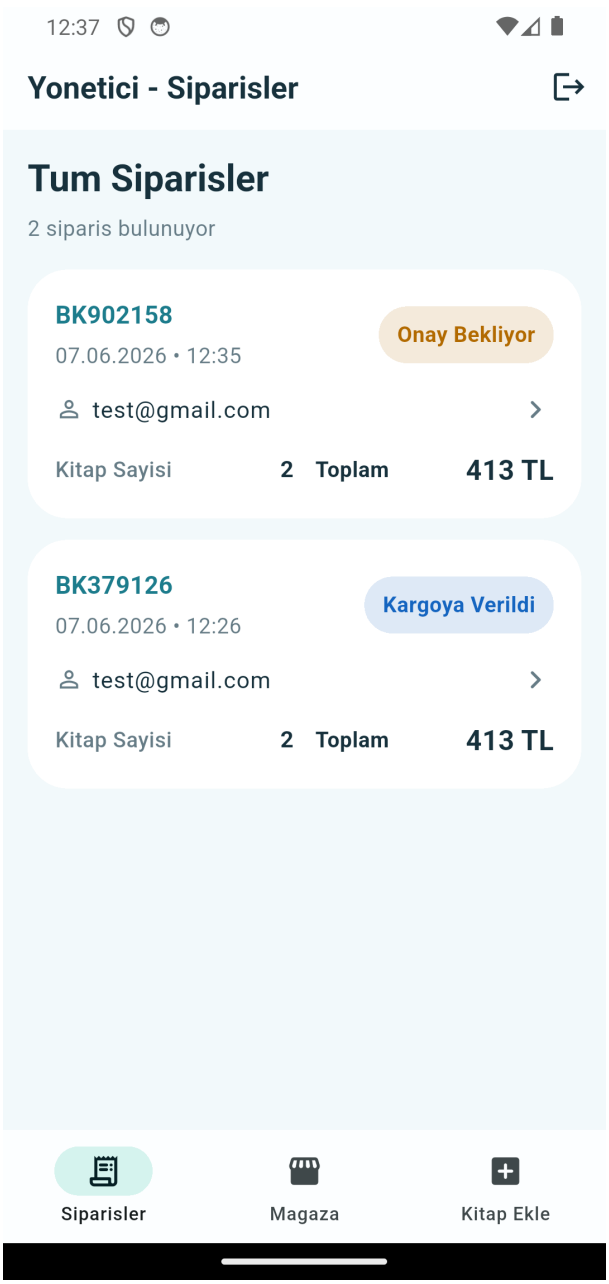


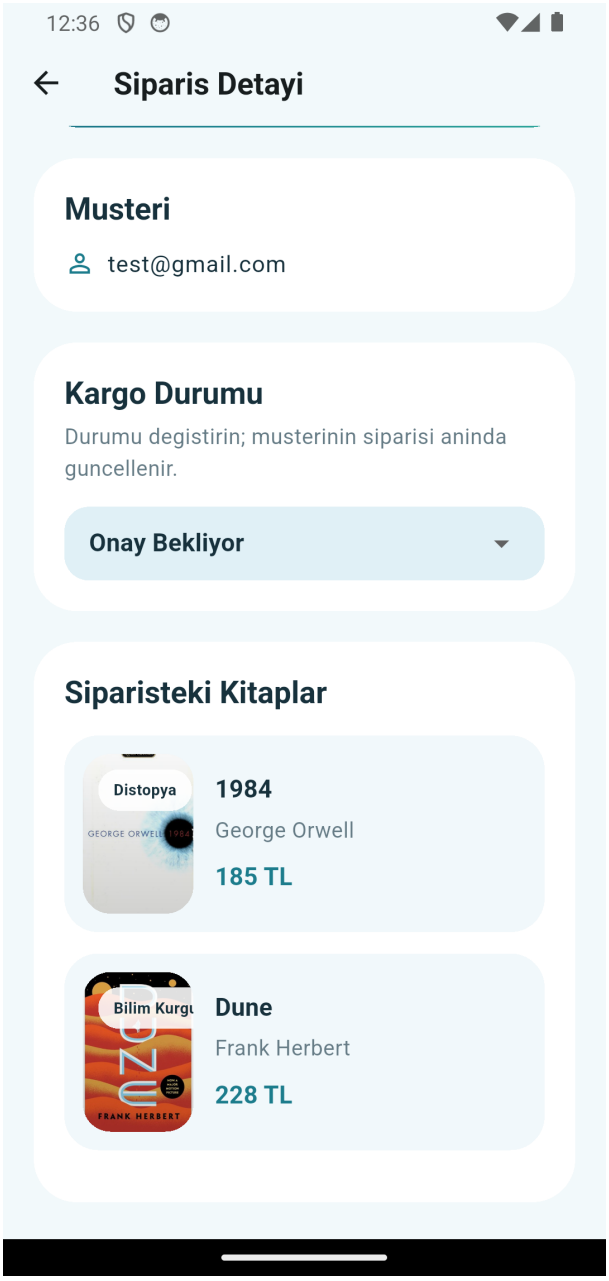
Figure 3: Ekran goruntusu 5



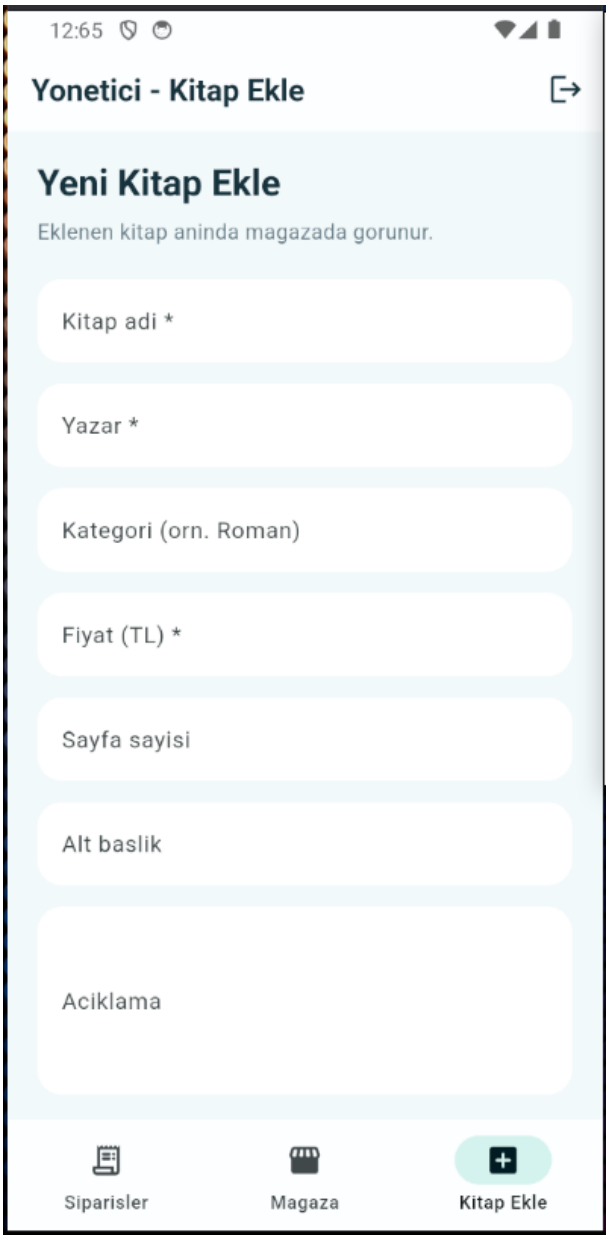
(a) Magaza ve admin onizleme



(b) Admin tum siparisler



(a) Admin siparis detayi



(b) Admin kitap ekleme formu

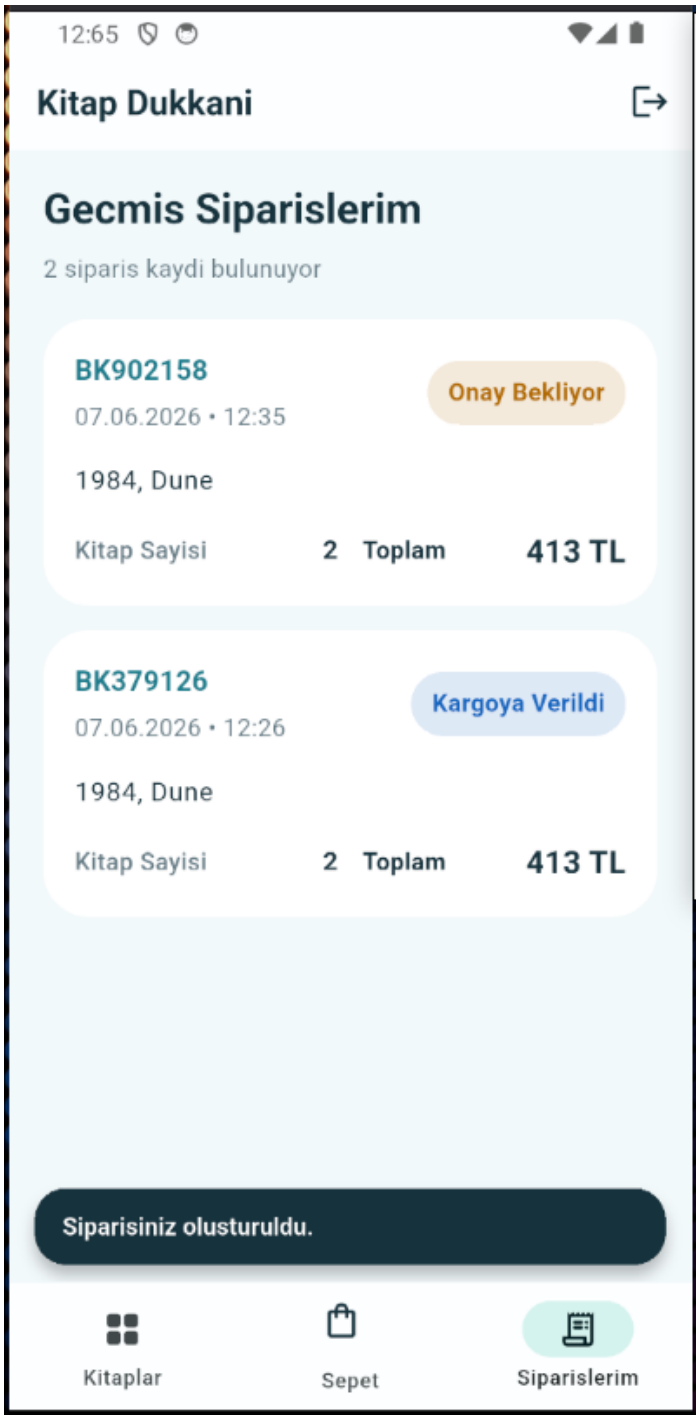


Figure 6: Musteri siparis durumlari